TP2 — Cryptographie R309 : Vigenère, ou la revanche d'Alice

Indice de coïncidence, chiffre de Vigenère et cryptanalyse automatique

AUTEUR
Johan Polsinelli

CLASSE
BUT Réseaux & Télécommunications

DATE
14 septembre 2026

Compte-rendu de travaux pratiques — dépôt cryptos/TP2 (responses.md)

PLAN DU DOCUMENT

Sommaire

Exercice 1 : Indice de coïncidence 3

1. Interprétation probabiliste de l'indice de coïncidence 3
2. Calcul manuel pour $m = \text{ABRACADABRA}$ 4
3. Texte très long avec distribution uniforme $(1/26)$ 4
4. Texte en français et fréquences $F(x)$ 5
5. Invariance par le chiffre de César 5
6. (Subsidiaire) Test de Kasiski sur le mot "DEMAIN" 6

Exercice 2 : Le chiffre de Vigenère 6

1. Chiffrement (`vigenere`) 6
2. Déchiffrement (`decode_vigenere`) 7
3. Indice de coïncidence en Python (`IC`) 8
4. Longueur de la clé (`key_length`) 8
5. Cryptanalyse automatique (`break_vigenere`) 10
6. Conclusion et analyse de sécurité 11

TP2 — Cryptographie R309 : Vigenère, ou la revanche d’Alice

Auteur : Johan Polsinelli
Date : 14 septembre 2026

Exercice 1 : Indice de coïncidence

Soit m un texte de longueur $n = |m|$ écrit sur l'alphabet $\mathcal{A}$. Pour chaque lettre $x \in \mathcal{A}$, on note n_x son nombre d'occurrences dans m ($\sum_{x \in \mathcal{A}} n_x = n$).
L'indice de coïncidence $\mathrm{IC}(m)$ est défini par :

$$\mathrm{IC}(m) := \frac{\sum_{x \in \mathcal{A}} n_x (n_x - 1)}{n(n-1)}$$

---

1. Interprétation probabiliste de l'indice de coïncidence

On choisit uniformément et au hasard deux positions distinctes (i, j) avec $i \neq j$ dans le texte m.

1. **Nombre total de tirages possibles :** Il y a n choix pour la première position et $(n-1)$ choix pour la seconde position, soit $n(n-1)$ couples ordonnés de positions distinctes.
2. **Nombre de cas favorables pour une lettre x fixée :** Pour que la première position contienne la lettre x et la seconde également, il faut choisir la première position parmi les n_x occurrences de x, puis la seconde parmi les $(n_x - 1)$ occurrences restantes. Il y a donc $n_x(n_x - 1)$ couples favorables.
3. **Probabilité d'obtenir la même lettre x :**

$$\mathbb{P}(\text{lettre}_1 = x \text{ et } \text{lettre}_2 = x) = \frac{n_x(n_x-1)}{n(n-1)}$$

4. **Probabilité globale :** Les événements « les deux lettres sont la lettre x » étant mutuellement exclusifs pour des lettres différentes, la probabilité que deux lettres tirées à des positions distinctes soient identiques est la somme sur tout l'alphabet :

$$\mathbb{P}(\text{deux lettres identiques}) = \frac{\sum_{x \in \mathcal{A}} n_x(n_x-1)}{n(n-1)} = \mathrm{IC}(m)$$

Conclusion : $\mathrm{IC}(m)$ représente **exactement** la probabilité que deux lettres tirées au hasard à des positions distinctes dans le texte m soient identiques.

2. Calcul manuel pour $m = \mathrm{ABRACADABRA}$

Le texte $m = \mathrm{ABRACADABRA}$ a pour longueur $n = 11$.

Décompte des occurrences n_x :

Lettre x	Positions	Occurrences n_x	Terme $n_x(n_x - 1)$
A	1, 4, 6, 8, 11	5	$5 \times 4 = 20$
B	2, 9	2	$2 \times 1 = 2$
R	3, 10	2	$2 \times 1 = 2$
C	5	1	$1 \times 0 = 0$
D	7	1	$1 \times 0 = 0$
Total	—	$n = 11$	**Numérateur** $= 24$

Calcul de $\mathrm{IC}(m)$:

- **Numérateur :** $\sum n_x(n_x - 1) = 20 + 2 + 2 + 0 + 0 = 24$
- **Dénominateur :** $n(n-1) = 11 \times 10 = 110$

$$\mathrm{IC}(m) = \frac{24}{110} = \boxed{\frac{\mathbf{12}}{\mathbf{55}} \approx \mathbf{0{,}2182}}$$

3. Texte très long avec distribution uniforme ($1/26$)

a) Expression de n_x en fonction de n

Si le texte est très long et que chaque lettre apparaît de manière équiprobable avec une fréquence de $1/26$:

$$n_x \approx \frac{n}{26}$$

b) Déduction de $\mathrm{IC}(m)$

Pour un texte très long, $n_x - 1 \approx n_x$ et $n - 1 \approx n$. On a donc :

$$\mathrm{IC}(m) \approx \sum_{x \in \mathcal{A}} \left(\frac{n_x}{n}\right)^2 \approx \sum_{x \in \mathcal{A}} \left(\frac{1}{26}\right)^2 = 26 \times \frac{1}{26^2} = \frac{1}{26}$$

$$\mathrm{IC}(\mathbf{m}) \approx \boxed{\frac{\mathbf{1}}{\mathbf{26}} \approx \mathbf{0{,}0385}}$$

4. Texte en français et fréquences $F(x)$

a) Démontrer que $\mathrm{IC}(m) \approx \sum_{x \in \mathcal{A}} F(x)^2$

Pour un texte français long, $n_x \approx nF(x)$. En réinjectant l'approximation dans la formule :

$$\mathrm{IC}(m) = \frac{\sum_{x \in \mathcal{A}} n_x(n_x - 1)}{n(n-1)} \approx \frac{\sum_{x \in \mathcal{A}} (nF(x))^2}{n^2} = \frac{n^2 \sum_{x \in \mathcal{A}} F(x)^2}{n^2} = \sum_{x \in \mathcal{A}} F(x)^2$$

$$\mathrm{IC}(\mathbf{m}) \approx \sum_{\mathbf{x} \in \mathcal{A}} \mathbf{F(x)^2}$$

b) Comparaison et conclusion

- **Texte aléatoire uniforme :** $\mathrm{IC} \approx 0{,}038$
- **Texte en français clair :** $\sum_{x \in \mathcal{A}} F(x)^2 \approx 0{,}078$

Interprétation : L'indice de coïncidence du français ($\approx 0{,}078$) est environ **deux fois supérieur** à celui d'un texte aléatoire. Cela s'explique par la non-uniformité de la langue française (la lettre 'E' est très fréquente, alors que 'W' ou 'K' sont rares). D'après l'inégalité de Cauchy-Schwarz, la somme des carrés est minimale pour la distribution uniforme et augmente avec l'irrégularité des fréquences.

5. Invariance par le chiffre de César

a) Comparaison de n_x et n_{x+k}

Un chiffre de César de clé k applique une permutation bijective sur l'alphabet : chaque lettre x est remplacée de façon déterministe par $x + k \pmod{26}$.
Le nombre d'occurrences de la lettre décalée dans le message chiffré m' est donc **strictement égal** au nombre d'occurrences de la lettre initiale dans m :

$$n_{x+k}(\text{dans } m') = n_x(\text{dans } m)$$

b) Démonstration de $\mathrm{IC}(m') = \mathrm{IC}(m)$

$$\mathrm{IC}(m') = \frac{\sum_{y \in \mathcal{A}} n'_y(n'_y - 1)}{n(n-1)} = \frac{\sum_{x \in \mathcal{A}} n'_{x+k}(n'_{x+k} - 1)}{n(n-1)} = \frac{\sum_{x \in \mathcal{A}} n_x(n_x - 1)}{n(n-1)} = \mathrm{IC}(m)$$

> **Propriété essentielle :** Le chiffre de César conserve exactement l'indice de coïncidence.

c) Analyse d'un message avec $\mathrm{IC} = 0{,}041$

Non, ce message n'a pas été chiffré par un chiffre de César.
Si le message avait été chiffré par un chiffre de César, son indice de coïncidence serait égal à celui du texte clair en français ($\approx 0{,}078$). Or, l'indice mesuré ($0{,}041$) est très proche de la valeur aléatoire ($0{,}0385$). Le chiffrement utilisé a donc lissé les fréquences, ce qui exclut tout chiffrement monoalphabétique.

6. (Subsidiaire) Test de Kasiski sur le mot "DEMAIN"

Deux occurrences du mot "DEMAIN" (6 lettres) sont chiffrées à l'identique aux positions 12 et 57.

a) Relation entre $57 - 12$ et la longueur de clé L

Pour que deux occurrences identiques soient chiffrées de la même manière, la clé doit être alignée à l'identique. La distance entre les deux positions doit donc être un multiple de la longueur de la clé L :

$$57 - 12 = 45 \implies \mathbf{L} \text{ divise } \mathbf{45}$$

b) Prise en compte d'un second écart de 30

La seconde répétition implique que L divise également 30 ($L \mid 30$).
Par conséquent, L doit diviser le $\mathrm{PGCD}(45, 30) = 15$.

$$\mathbf{L} \in \{\mathbf{1}, \mathbf{3}, \mathbf{5}, \mathbf{15}\}$$

(Si l'on sait par ailleurs que le texte n'est pas un chiffre de César monoalphabétique, on peut exclure $L = 1$).

Exercice 2 : Le chiffre de Vigenère

1. Chiffrement (`vigenere`)

La fonction `vigenere(msg_clear, key_word)` nettoie le message et la clé, puis applique le décalage périodique :

$$\mathrm{Code}[i] = (\mathrm{Clair}[i] + \mathrm{Clé}[i \bmod L]) \pmod{26}$$

```python
def vigenere(msg_clear, key_word):
    m = clean_message(msg_clear)
    k = clean_message(key_word)
    res = []
    len_k = len(k)
    for i in range(len(m)):
        shift = ord(k[i % len_k]) - ord("A")
        cipher_char = chr((ord(m[i]) - ord("A") + shift) % 26 + ord("A"))
        res.append(cipher_char)
    return "".join(res)
```

- **Vérification :** `vigenere("Salut Alice", "Bob")` renvoie bien `"TOMVHBMWDF"`.

2. Déchiffrement (`decode_vigenere`)

Pour déchiffrer, on soustrait le décalage de la clé :

$$\text{Clair}[i] = (\text{Code}[i] - \text{Clé}[i \bmod L]) \pmod{26}$$

```python
def decode_vigenere(msg_coded, key_word):
    c = clean_message(msg_coded)
    k = clean_message(key_word)
    res = []
    len_k = len(k)
    for i in range(len(c)):
        shift = ord(k[i % len_k]) - ord("A")
        clear_char = chr((ord(c[i]) - ord("A") - shift) % 26 + ord("A"))
        res.append(clear_char)
    return "".join(res)
```

Contenu de `message bob.txt` (chiffré avec la clé `"BOB"`) :

En appliquant `decode_vigenere` au fichier `message bob.txt` avec la clé `"BOB"`, on obtient le message clair rédigé par Bob :

```
RENDEZVOUSDEMAINALARECREDERRIERELEGYMNASESURTOUTNEDISRIENAPERSONNE
```

> **Version lisible :** « Rendez-vous demain à la récré, derrière le gymnase. Surtout, ne dis rien à personne. »

3. Indice de coïncidence en Python (`IC`)

```python
from collections import Counter


def IC(message):
    m = clean_message(message)
    n = len(m)
    if n <= 1:
        return 0.0
    counts = Counter(m)
    return sum(nx * (nx - 1) for nx in counts.values()) / (n * (n - 1))
```

Analyse du fichier `message alice.txt` :

- **Valeur mesurée :** $\mathrm{IC}(\mathrm{alice}) \approx \mathbf{0{,}0470}$
- **Déduction :** L'indice étant proche de $0{,}038$ et très éloigné de $0{,}078$, le message d'Alice a été chiffré par un chiffrement polyalphabétique (Vigenère) avec un mot-clé de longueur $L > 1$.

4. Longueur de la clé (`key_length`)

En découpant le texte en L colonnes (`msg[i::L]`), chaque colonne regroupe les lettres chiffrées par le même décalage de la clé. Si L est la bonne longueur, chaque colonne est un simple chiffre de César et aura un $\mathrm{IC} \approx 0{,}078$.

```python
from tabulate import tabulate


def key_length(msg_coded, lmax=20):
    m = clean_message(msg_coded)
    smallest_L = None
    table_data = []

    for L in range(1, lmax + 1):
        columns = [m[i::L] for i in range(L)]
        avg_ic = sum(IC(col) for col in columns) / L
        table_data.append([L, f"{avg_ic:.4f}"])

        if avg_ic > 0.06 and smallest_L is None:
            smallest_L = L

    print(
        tabulate(
            table_data, headers=["Longueur L", "IC Moyen"], tablefmt="fancy_grid"
        )
    )
    return smallest_L
```

Résultats pour le message d'Alice :

Longueur L	IC Moyen
1	0.047
2	0.0467
3	0.0479
4	0.0478
5	0.0809
6	0.0482
7	0.0462
8	0.0468
9	0.0482
10	0.0796
11	0.05
12	0.0484
13	0.0448
14	0.0445
15	0.0831
16	0.0477
17	0.0448
18	0.0478
19	0.0451
20	0.0832

Lecture du tableau :

- $L \in \{1, 2, 3, 4, 6, 7, 8, 9, 11, \ldots, 19\}$: $\mathrm{IC} \approx 0{,}045$ à $0{,}05$, c'est-à-dire le niveau d'un texte à fréquences **lissées** — ce ne sont pas les bonnes longueurs.
- $L = 5$: $\mathrm{IC} \approx 0{,}0809$ → le seuil de $0{,}06$ est franchi, chaque colonne redevient un simple chiffre de César sur du français.
- $L = 10, 15, 20$ (multiples de 5) : $0{,}0796$, $0{,}0831$, $0{,}0832$ → également au-dessus du seuil, ce sont les harmoniques de la période réelle.

- **Longueur détectée : $\mathbf{L = 5}$** (première valeur — donc la plus petite — franchissant le seuil de $0{,}06$)

- **Pourquoi la plus petite valeur et non le maximum ?**
  Tout multiple de la vraie longueur ($2L = 10$, $3L = 15$, $4L = 20$) découpe le texte en sous-colonnes qui subissent elles aussi un décalage unique, donnant également un IC très élevé ($\approx 0{,}08$). Ici, $L = 15$ ($0{,}0831$) et $L = 20$ ($0{,}0832$) obtiennent même un IC moyen **supérieur** à celui de la période fondamentale ($0{,}0809$), à cause des fluctuations statistiques sur des colonnes plus courtes. On sélectionne donc la **plus petite valeur de L** franchissant le seuil, seule garantie d'obtenir la période réelle de la clé.

5. Cryptanalyse automatique (`break_vigenere`)

```
def load_frequencies(filename="frequences francais.txt"):
    freq = {}
    with open(filename, "r", encoding="utf-8") as f:
        for line in f:
            if line.strip():
                letter, value = line.split()
                freq[letter.upper()] = float(value)
    return freq


FREQ_FR = load_frequencies()


def break_cesar(column):
    best_score, best_shift = -1, 0
    n = len(column)
    counts = Counter(column)
    for shift in range(26):
        score = sum(
            (count / n)
            * FREQ_FR.get(chr((ord(c) - ord("A") - shift) % 26 + ord("A")), 0)
            for c, count in counts.items()
        )
        if score > best_score:
            best_score, best_shift = score, shift
    return chr(ord("A") + best_shift)


def break_vigenere(msg_coded):
    m = clean_message(msg_coded)
    L = key_length(m)
    key_letters = [break_cesar(m[i::L]) for i in range(L)]
    key_word = "".join(key_letters)
    return key_word, decode_vigenere(m, key_word)
```

Résultats pour le message d'Alice :

- **Mot-clé trouvé :** `ALICE`

- **Sortie du programme :**

```
================================================
MOT-CLÉ TROUVÉ : ALICE
================================================

MESSAGE DÉCHIFFRÉ :

BOBJECROISQUENOTREENSEIGNANTAREUSSIALIRENOSMESSAGESPRECEDENTSILFAUTCHANGERDEMETHODE
CARLEDECALAGEUNIQUENERESISTEARIENDUTOUTJAITROUVEDANSUNVIEUXLIVREUNPROCEDEATTRIBUEAU
NDIPLOMATEITALIENQUICONSISTEAUTILISERUNMOTSECRETPLUTOTQUUNSIMPLENOMBREONECRITLEMOTS
OUSLEMESSAGEENLEREPETANTAUTANTDEFOISQUENECESSAIREPUISONDECALECHAQUELETTREDUCLAIRPAR
LALETTREDELACLEQUISETROUVEJUSTEENDESSOUSAINSIUNEMEMELETTREDUMESSAGENEDONNEPLUSTOUJO
URSLAMEMELETTRECODEECEQUIDEVRAITRUINERCOMPLETEMENTSONANALYSEDESFREQUENCESRETIENSBIE
NLEMOTSECRETETSURTOUTNELECRISNULLEPARTSITUASCOMPRISREPONDSMOIAVECLAMEMEMETHODEETNOU
SPOURRONSCONTINUERTRANQUILLEMENT
```

- **Version lisible :**
  « Bob, je crois que notre enseignant a réussi à lire nos messages précédents. S'il faut changer de méthode, car le décalage unique ne résiste à rien du tout. J'ai trouvé dans un vieux livre un procédé attribué à un diplomate italien qui consiste à utiliser un mot secret plutôt qu'un simple nombre. On écrit le mot sous le message en le répétant autant de fois que nécessaire puis on décale chaque lettre du clair par la lettre de la clé qui se trouve juste en dessous. Ainsi une même lettre du message ne donne plus toujours la même lettre codée, ce qui devrait ruiner complètement son analyse des fréquences. Retiens bien le mot secret et surtout ne l'écris nulle part. Si tu as compris, réponds-moi avec la même méthode et nous pourrons continuer tranquillement. »

- **Contrôle par l'indice de coïncidence :** sur le texte déchiffré, $\mathrm{IC} \approx 0{,}0808$, soit la valeur attendue pour du français littéral ($\approx 0{,}078$) contre $0{,}0470$ sur le chiffré : les fréquences de la langue sont bien revenues, la clé trouvée est la bonne.

- **Lecture du message :** Alice constate que le chiffrement par décalage unique (César) a été cassé, et propose à Bob de passer au **procédé attribué à un diplomate italien** — le chiffre de Vigenère, historiquement attribué à tort au Français Blaise de Vigenère alors qu'il est dû à l'Italien Giovan Battista Bellaso. Elle lui demande de répondre avec la même méthode : les deux personnages échangent donc via un mot-clé secret, qu'Alice choisit être son propre prénom.

6. Conclusion et analyse de sécurité

a) Coût d'une attaque par force brute

Un mot-clé de longueur L sur un alphabet de 26 lettres offre 26^L clés possibles :

- **Pour $L = 5$:** $26^5 = 11\,881\,376 \approx \mathbf{1{,}2 \times 10^7}$ clés
- **Pour $L = 10$:** $26^{10} = 141\,167\,095\,653\,376 \approx \mathbf{1{,}4 \times 10^{14}}$ clés

b) Coût de notre attaque statistique

Grâce à l'indice de coïncidence et à l'analyse fréquencielle par colonne, nous n'avons testé que :

$$\text{Essais} = l_{\max} + L \times 26 = 20 + 5 \times 26 = \mathbf{150} \text{ opérations pour } \mathbf{L = 5}$$

(Pour $L = 10$, il suffit de $20 + 10 \times 26 = 280$ opérations).

c) Le chiffre de Vigenère est-il sûr ?

Non, le chiffre de Vigenère n'est pas sûr. Sa complexité d'attaque n'est pas exponentielle (26^L) mais seulement linéaire ($L \times 26$). L'indice de coïncidence permet de découpler le problème global en L sous-problèmes de César indépendants, car la répétition de la clé conserve les propriétés statistiques de la langue source au sein des colonnes.

d) Conditions pour le rendre invulnérable (Le Masque Jetable / Le chiffre de Vernam)

Pour rendre ce chiffrement incassable (sécurité parfaite au sens de Shannon), il faut respecter les 3 conditions du **masque jetable** (*One-Time Pad*) :

1. **Longueur de clé :** La clé doit être **aussi longue que le message** ($L = n$). Il n'y a plus de périodicité ni de colonnes.
2. **Aléatoire parfait :** La clé doit être tirée selon une loi **uniforme et indépendante** (pas un mot du dictionnaire).
3. **Usage unique :** Chaque clé ne doit être utilisée **qu'une seule fois** puis détruite.